TV-SIDARHE Model

Algorithm Technical Documentation

A Compartmental Model of Infectious Disease Transmission

Language: Python
Algorithm: TV-SIDARHE (Time-Varying SIDARHE)
Date: March 2026

1. Overview

This document describes the algorithmic implementation of the TV-SIDARHE calibration and prediction pipeline. The script reads observed compartmental data, fits the TV-SIDARHE continuous-time ODE system to a training set using nonlinear least squares, and extrapolates the fitted trajectory over a combined training and test horizon.

Unlike the discrete-time forward Euler formulation used in the class-based TV-SIDARHE implementation, this pipeline integrates the ODE system numerically using a fourth-order Runge-Kutta (RK4) scheme, which provides higher numerical fidelity and enables continuous-time parameterisation. The fitted parameters are subsequently used to generate predictions for evaluation against held-out test data.

Module	Role
Data ingestion	Load training and test observations from CSV files
ODE system (SIDARHE)	Define TV-SIDARHE continuous-time dynamics with time-varying b^{-I} and $\Phi(t)$
RK4 integrator (rks4)	Numerically solve the ODE system over a specified time horizon
Residual functions (rfun, rfun1)	Compute weighted fit error between model trajectory and observed data
Parameter optimisation	Minimise residuals via bounded least-squares to recover calibrated parameters

Module	Role
Prediction and output	Extend trajectory to full horizon; compute R^2 and MAPE; export predictions

2. Data Ingestion

2.1 Input Files

The pipeline requires two CSV files: `traindata.csv` and `testdata.csv`. Each file must contain columns `D`, `R`, `H`, and `E`, corresponding to the four observable compartments of the TV-SIDARHE model. The Susceptible (S), Infected (I), and Ailing (A) compartments are treated as latent states and are not directly observed.

Variable	Compartment	Status in Calibration
`D`	Diagnosed	Observable; included in training data
`R`	Recognised	Observable; used as the calibration target (weight = 1)
`H`	Healed	Observable; excluded from loss by default (weight = 0)
`E`	Extinct	Observable; excluded from loss by default (weight = 0)

Note: The weight vector `[0, 1, 0, 0]` in `rfun` causes the optimiser to fit only the Recognised (R) compartment trajectory. The other three observable compartments contribute zero residual during calibration. This choice concentrates the optimisation signal on the compartment deemed most reliably observed.

2.2 Array Construction

After loading, each compartment column is extracted as a NumPy array and stacked column-wise into an observation matrix:

```
y   = np.transpose(np.array([D, R, H, E]))    # shape: (n_train, 4)
y_t = np.transpose(np.array([D_t, R_t, H_t, E_t]))    # shape: (n_test, 4)
pre_long = y.shape[0] + y_t.shape[0]          # total simulation horizon
```

The combined length `pre_long` defines the total number of time steps over which the ODE is integrated after calibration, spanning both training and test periods in a single continuous forward pass.

3. ODE System: SIDARHE Function

3.1 Function Signature

The function `SIDARHE(t, y, arg, N)` defines the right-hand side of the TV-SIDARHE ODE system in continuous time. It takes the current time `t`, the current state vector `y` (length 7), the parameter vector `arg` (length 11), and the total population `N`. It returns the vector of time derivatives `[dS, dI, dA, dD, dR, dH, dE]`.

3.2 State Vector Layout

Index	Variable	Compartment
y[0]	`S`	Susceptible
y[1]	`I`	Infected (undetected)
y[2]	`A`	Ailing (undetected)
y[3]	`D`	Diagnosed
y[4]	`R`	Recognised
y[5]	`H`	Healed
y[6]	`E`	Extinct

3.3 Parameter Vector Layout

Index	Symbol	Interpretation	Bounds
arg[0]	α	Baseline transmission rate from I	[0, 1]
arg[1]	β	Baseline transmission rate from A	[0, 1]
arg[2]	ε	Baseline diagnosis rate (I → D)	[0, 1]
arg[3]	ζ	Symptom onset / D → R progression rate	[0, 1]
arg[4]	θ	Baseline recognition rate (A → R)	[0, 1]
arg[5]	ρ	Recovery rate from I	[0, 1]
arg[6]	σ	Recovery rate from A	[0, 1]
arg[7]	κ	Recovery rate from R	[0, 1]
arg[8]	λ	Recovery rate from D	[0, 1]
arg[9]	a	Epidemic detection capacity (log base)	[0, ∞)
arg[10]	b	Government intervention intensity (decay base)	[0, ∞)

Note: The mortality parameter μ is fixed at 10^{-10} within the function body and is not included in the

optimised argument vector. It is treated as a near-zero constant, effectively removing mortality as a calibrated degree of freedom in this implementation.

3.4 Time-Varying Terms

Two time-dependent modifiers are evaluated at each call to the ODE function, using the continuous time variable t:

$$\Lambda(t) = b^{-I} \cdot [\alpha \cdot I(t) + \beta \cdot A(t)] \quad \text{(force of infection)}$$

$$\Phi(t) = \log_a(t + 10^{-10}) + 1 \quad \text{(detection capacity growth factor)}$$

Because `t` is a continuous variable in this formulation, both modifiers vary smoothly across integration sub-steps, unlike the discrete-time implementation where they are updated only at integer step boundaries. The offset 10^{-10} prevents a domain error when `t → 0`.

3.5 Full ODE Equations

```
dS = -(b**(-t)*α*I + b**(-t)*β*A) * S / N
dI =  (b**(-t)*α*I + b**(-t)*β*A) * S / N - ((Φ(t)*ε) + ζ + ρ) * I
dA =  ζ*I - (Φ(t)*θ + σ) * A
dD =  Φ(t)*ε*I - (ζ + λ) * D
dR =  ζ*D + Φ(t)*θ*A - (κ + μ) * R
dH =  ρ*I + σ*A + λ*D + κ*R
dE =  μ*R
```

The population normalisation factor `N` appears in the `dS` and `dI` equations, consistent with a frequency-dependent (proportionate mixing) transmission formulation. All other transitions are density-independent linear flows between compartments.

4. Numerical Integrator: rks4

4.1 Algorithm

The function `rks4(f, a, b, Za, M)` implements the classical fourth-order Runge-Kutta method over the interval [a, b] with M uniform steps. It returns the time grid and the full state trajectory as a matrix of shape (M+1) × 7.

Argument	Type	Description
`f`	Callable	ODE right-hand side: f(t, y) → dy/dt
`a`	float	Start of integration interval (set to 1 in all calls)
`b`	float/int	End of integration interval

Argument	Type	Description
`Za`	ndarray	Initial state vector at t = a (length 7)
`M`	int	Number of integration steps; step size h = (b − a) / M

```
h  = (b - a) / M
for i in range(1, M+1):
    k1 = h * f(t[i-1],          Z[i-1,:])
    k2 = h * f(t[i-1] + h/2,    Z[i-1,:] + k1/2)
    k3 = h * f(t[i-1] + h/2,    Z[i-1,:] + k2/2)
    k4 = h * f(t[i-1] + h,      Z[i-1,:] + k3)
    Z[i,:] = Z[i-1,:] + (k1 + 2*k2 + 2*k3 + k4) / 6
```

The step size `h = (b - a) / M` is set so that M = len − 1 and [a, b] = [1, len], yielding `h = 1` for all calibration calls. For the full prediction run, the same unit step size is preserved: [a, b] = [1, pre_long], M = pre_long − 1.

5. Initial Conditions

The initial state vector is constructed from the first observed data point and two fixed seed values for the latent compartments I and A:

```
S  = N - y[0][0] - y[0][1] - y[0][2] - y[0][3] - 20000 - 10000
y0 = np.array([S, 20000, 10000, y[0][0], y[0][1], y[0][2], y[0][3]])
#                    I0      A0        D0          R0          H0          E0
```

Compartment	Initial Value	Source
I_0 (Infected)	20,000	Fixed seed value
A_0 (Ailing)	10,000	Fixed seed value
D_0 (Diagnosed)	`y[0][0]`	First row of training data
R_0 (Recognised)	`y[0][1]`	First row of training data
H_0 (Healed)	`y[0][2]`	First row of training data
E_0 (Extinct)	`y[0][3]`	First row of training data
S_0 (Susceptible)	$N - D_0 - R_0 - H_0 - E_0 - 20000 - 10000$	Conservation identity

The values $I_0 = 20{,}000$ and $A_0 = 10{,}000$ are fixed constants in the current implementation. S_0 is derived by subtracting all other compartments from the total population N = 140,005 × 10^4, ensuring the conservation identity $S + I + A + D + R + H + E = N$ holds at t = 0.

6. Residual and Loss Functions

6.1 rfun — Vectorised Residual (used by least_squares)

The function `rfun(args, ydata, N)` integrates the ODE over the training interval and returns a residual vector of length equal to the number of training time steps. Each element is the weighted absolute error between the model trajectory and the observed data at that step.

```
def rfun(args, ydata, N):
    n = ydata.shape[0]
    _, yval = rks4(lambda t,y: SIDARHE(t,y,args,N), 1, n, y0, n-1)
    weights = np.array([0, 1, 0, 0])   # fit R only
    return np.sum(np.abs(yval[:,3:] - ydata) * weights, axis=1)
```

The slice `yval[:,3:]` extracts the four observable compartments (D, R, H, E) from the full 7-column state matrix. The weight vector selects only R (index 1 within the slice) as the calibration target. The returned vector has shape (n_train,) and is passed directly to `scipy.optimize.least_squares`, which minimises its sum-of-squares norm.

6.2 rfun1 — Scalar Loss (auxiliary)

The auxiliary function `rfun1(arg)` computes a single scalar loss value: the total weighted absolute deviation across all training steps. It uses a column weight vector and a matrix dot product, equivalent to `rfun` but returning a scalar sum rather than a per-step vector. This function is defined but not invoked by the optimisation loop in the current pipeline; it may be used for manual diagnostics or gradient-free search.

7. Parameter Optimisation

7.1 Solver Configuration

Parameters are estimated by calling `scipy.optimize.least_squares` with the vectorised residual function `rfun`. The solver minimises the sum of squared residuals using a trust-region reflective algorithm.

```
arg0 = np.array([0.11, 0.105, 0.143, 0.57, 0.196,
                 0.011, 0.017, 0.371, 0.456, 3, 3])

optiargs = least_squares(
    lambda arg: rfun(arg, y, N),
    arg0,
    bounds = ((0,0,0,0,0,0,0,0,0,0,0),
              (1,1,1,1,1,1,1,1,1,np.inf,np.inf)),
    max_nfev = 10000
)
```

Setting	Value	Notes
Initial parameters (arg0)	See code	Epidemiologically plausible starting point; influences convergence
Lower bounds	0 for all α–λ; 0 for a, b	Prevents negative rates and degenerate bases
Upper bounds	1 for α–λ; ∞ for a, b	Rates capped at 1 per unit time; a and b unbounded above
Max function evaluations	10,000	Termination criterion; increase if convergence is not achieved
Method	Trust Region Reflective (default)	Handles box-constrained problems; gradient estimated by finite differences

7.2 Optimised Parameters

On convergence, the calibrated parameter vector is stored in `optiargs.x`. The full optimisation result object `optiargs` is printed to console and contains the final residual vector, cost, Jacobian, and convergence status. The optimised parameters are then used in a single forward RK4 integration over the full `pre_long` horizon to produce both training-period fit and test-period predictions.

8. Prediction and Evaluation

8.1 Full-Horizon Forward Pass

After calibration, the ODE is integrated once over the combined training and test horizon using the optimised parameters:

```
tval, yval = rks4(
    lambda t, y: SIDARHE(t, y, optiargs.x, N),
    1, pre_long, y0, pre_long - 1
)
```

The resulting matrix `yval` has shape (pre_long, 7). The training portion corresponds to rows `0 : n_train` and the prediction portion to rows `n_train : pre_long`. Observable compartments are extracted via `yval[:,3:]` (columns D, R, H, E).

8.2 Evaluation Metrics

Metric	Formula	Applied to
R^2 (coefficient of determination)	`r2_score(yval[:n,4], y[:,1])`	Training period, R compartment
MAPE (mean absolute	`mean(|y_pred − y_true| / y_true)`	Test period, R

Metric	Formula	Applied to
percentage error)		compartment

Both metrics target column index 4 of `yval` (the R compartment in the full state matrix), matched against column index 1 of the observed arrays `y` and `y_t` respectively.

8.3 Visualisation

The script renders a single Matplotlib figure overlaying three curves on the same axes:

Curve	Line Style	Data Source	Time Range
Fit	Red dashed	`yval[:,4]`	Full horizon (training + test)
True (training)	Black dashed	`y[:,1]`	Training period
Test (held-out)	Yellow solid	`y_t[:,1]`	Test period only

The x-axis label is `'days'`. The legend is placed at the best available position. MAPE on the test set and R^2 on the training set are computed and stored in `R2` and `err` respectively, but are not annotated on the figure in the current implementation.

9. Post-Processing and Output

9.1 Prediction Array

The test-period predictions for the four observable compartments are extracted and written to `y_final.csv`:

```
y_predict = yval[y.shape[0]:, 3:]   # shape: (n_test, 4)
np.savetxt('y_final.csv', y_predict, delimiter=',')
```

9.2 Normalisation Variants

The full concatenated trajectory (training + prediction) is assembled and three normalised variants are computed for downstream use:

Variable	Method	Formula
`y_final1`	Min-max (mean-centred)	$(x - \mu) / (x_max - x_min)$
`y_final2`	Z-score (Bessel-corrected)	$(x - \mu) / s$
`y_final3`	Robust scaling	Median/IQR via sklearn RobustScaler

For `y_final1` and `y_final2`, the mean, min, max, and standard deviation are computed column-wise from `y_final` (the full combined array), then broadcast to a constant-row matrix of the same shape. The standard deviation uses Bessel's correction ($n / (n - 1)$ scaling from variance). For `y_final3`, sklearn's `RobustScaler` is fit on the combined array, using median and interquartile range.

10. Dependencies and Execution

10.1 Required Packages

Package	Usage
numpy	Array operations, ODE state matrix, savetxt
pandas	CSV ingestion
scipy.optimize.least_squares	Bounded nonlinear parameter calibration
sklearn.metrics.r2_score	Training-period goodness-of-fit
sklearn.preprocessing.RobustScaler	Robust normalisation of output
matplotlib / seaborn	Trajectory visualisation
math	math.log used inside SIDARHE for scalar log evaluation
time	Wall-clock timing of the full pipeline

10.2 Execution

The script runs end-to-end as a single `.py` file. Place `traindata.csv` and `testdata.csv` in the working directory and execute:

```
python TVSIDARHE.py
```

References

[1] Giordano, G., Blanchini, F., Bruno, R., Colaneri, P., Di Filippo, A., Di Matteo, A., & Colaneri, M. (2020). Modelling the COVID-19 epidemic and implementation of population-wide interventions in Italy. *Nature Medicine*, 26(6), 855–860.

[2] Virtanen, P., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17, 261–272.

[3] Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

[4] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.

[5] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.